

SonarQube with Jenkins

SonarQube

SonarQube is a code quality analysis tool it automatically analyzes source code to detect,

- **Bugs** (A bug is something that can cause your code to behave incorrectly)

```
def divide(a, b):  
    return a / b # ✗ Risk of ZeroDivisionError
```

- **Code smells** (These aren't bugs but poor practices that make code harder to read, maintain, or extend.)

```
public void processOrder(Order order) {  
    // 50+ lines of code inside one method  
}  
// ✗ Large method: hard to test, read, or reuse (code smell)
```

- **Duplicated code**

- **Vulnerabilities** (These are issues that could be exploited by an attacker.)

```
String password = "myPassword123";  
// ✖ Hardcoded credentials – a common security issue
```

- **Complex code structures**

```
if (a > b) {  
    if (c > d) {  
        if (e < f) {  
            // ✖ Deeply nested logic – difficult to read and test  
        }  
    }  
}
```

```
if (condition1 && condition2 || condition3 && !condition4 || (condition5 && condition6)) {  
    // ✖ Complex boolean logic – needs simplification or comments  
}
```

Jenkins

Jenkins is an open-source automation server used primarily for Continuous Integration (CI) and Continuous Delivery (CD) in software development.

Requirements

- JDK 17 or 21 ([https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe \(sha256\)](https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)))
- Jenkins (<https://www.jenkins.io/download/>)
- SonarQube ([SonarQube Download Success | Community Build | Sonar](#))
- Apache Maven (<https://dlcdn.apache.org/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.zip>)

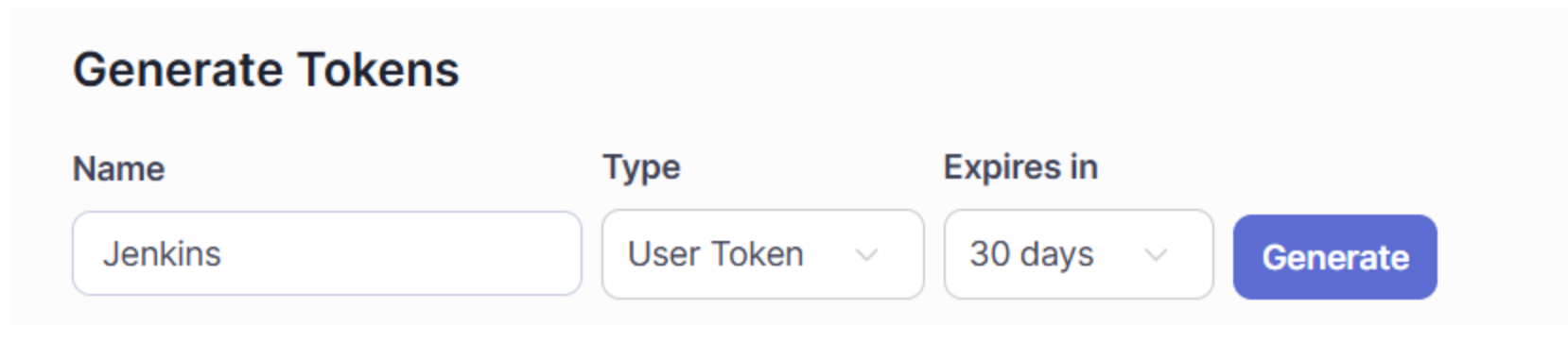
Install SonarQube

- Download and extract
- Set environment variable **SONAR_JAVA_PATH** as C:\Program Files\Java\jdk-21\bin\java.exe
- Run as Admin **StartSonar.bat**
- **Open** <http://localhost:9000/>
- Login as : admin and password : admin then change the password

Connect SonarQube with Jenkins

1. Generate SonarQube user token

My Account --> security --> Generate Tokens and generate a token(can give any name as token name)
Copy the token **squ_ea87de659097d6d935027a914d704d0cbebbde61**



The screenshot shows the 'Generate Tokens' interface in SonarQube. It features three input fields: 'Name' with the value 'Jenkins', 'Type' with a dropdown menu showing 'User Token', and 'Expires in' with a dropdown menu showing '30 days'. A blue 'Generate' button is positioned to the right of these fields.

Name	Type	Expires in
Jenkins	User Token	30 days

2. Add user token to Jenkins as a credential

Dashboard --> Manage Jenkins --> Credentials --> Global --> Add Credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

jenkins-sonar

Description ?

jenkins sonar user token

Create

Secret = copied user token and ID and Description can be anything.

3. Install the below plugins to Jenkins

Dashboard --> Manage Jenkins --> Plugins -- > Available Plugins

- Pipeline Stage View
- Maven Integration
- SonarQube Scanner

4. Add the below tools to Jenkins

Dashboard --> Manage Jenkins --> Tools

- JDK installations
- SonarQube Scanner installations
- Maven installations

The paths should be set as your computer locations of the applications.
(JAVA_HOME, SONAR_RUNNER_HOME...)

JDK installations

Add JDK

≡ **JDK**

Name

jdk21

JAVA_HOME

C:\Program Files\Java\jdk-21

☐ Install automatically ?

Add JDK

SonarQube Scanner installations

Add SonarQube Scanner

≡ **SonarQube Scanner**

Name

sonar

SONAR_RUNNER_HOME

C:\SonarQube\sonarqube-25.7.0.110598

☐ Install automatically ?

Add SonarQube Scanner

Maven installations

Add Maven

≡ **Maven**

Name

maven3

MAVEN_HOME

C:\Maven\apache-maven-3.9.11

☐ Install automatically ?

Add Maven

5. Add SonarQube server to Jenkins

Dashboard --> Manage Jenkins --> System

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

sonarqube-server

Server URL

Default is `http://localhost:9000`

`http://localhost:9000`

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

jenkins sonar user token

+ Add

Advanced ▾

Add SonarQube

Save Apply

- Name can be any name. But use in future.
- Server URL is the SonarQube local URL
- Snanner authentication token is user token (saved at step 2)

6. Create a pipeline

Dashboard--> New Item

New Item

Enter an item name

jenkins-sonar-project-1

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, runs a shell or a Java class, and performs post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project. Jenkins takes advantage of your POM files and Maven's dependency management to build your project.



Pipeline

Orchestrates long-running activities that can span multiple build agents (e.g., Jenkins, Docker, Kubernetes) and/or organizing complex activities that do not easily fit into a Freestyle project.



Multi-configuration project

Suitable for projects that need a large number of different configurations (e.g., different operating systems, different Java versions, platform-specific builds, etc.).



Folder

Creates a container that stores nested items in it. Useful for grouping related items. Each folder creates a separate namespace, so you can have multiple things with the same name in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in a repository.

OK

7. generate the git code

Dashboard --> jenkins-sonar-project-1 --> Pipeline Syntax

Steps

Sample Step

checkout: Check out from version control

checkout ?

SCM

Git

Repositories ?

Repository URL ?

https://github.com/kasunkosala/jdk-mvn-git-sonar-docker-jenk-cicd

Credentials ?

- none -

+ Add

Advanced ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

- Repository URL is the our java code containing GitHub URL.
- Change the Branch Spiffier from master to main.
- Then click "**Generate Pipeline Script**" and copy the code.

```
checkout scmGit(branches: [[name: '*/main']], extensions: [],  
userRemoteConfigs: [[url: 'https://github.com/kasunkosala/jdk-  
mvn-git-sonar-docker-jenk-cicd']])
```

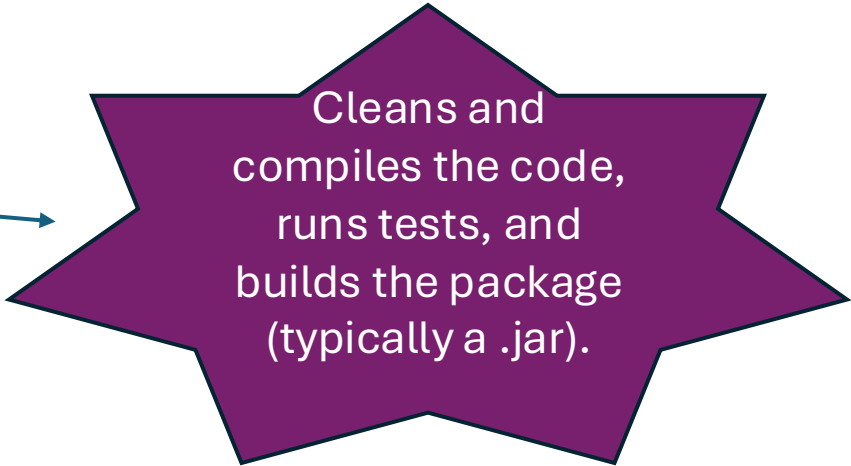
8. Generate the final code

```
pipeline {  
  
    agent any  
  
    tools {  
        jdk 'jdk21'  
        maven 'maven3'  
    }  
  
    environment {  
        SONARQUBE_SERVER = 'sonarqube-server'  
    }  
  
    stages {  
        stage('Checkout') {  
            steps {  
                checkout scmGit(branches: [[name: '*/main']], extensions: [],  
userRemoteConfigs: [[url: 'https://github.com/kasunkosala/jdk-mvn-git-sonar-docker-jenk-cicd']])  
            }  
        }  
    }  
}
```



The GitHub repository is cloned into the workspace.

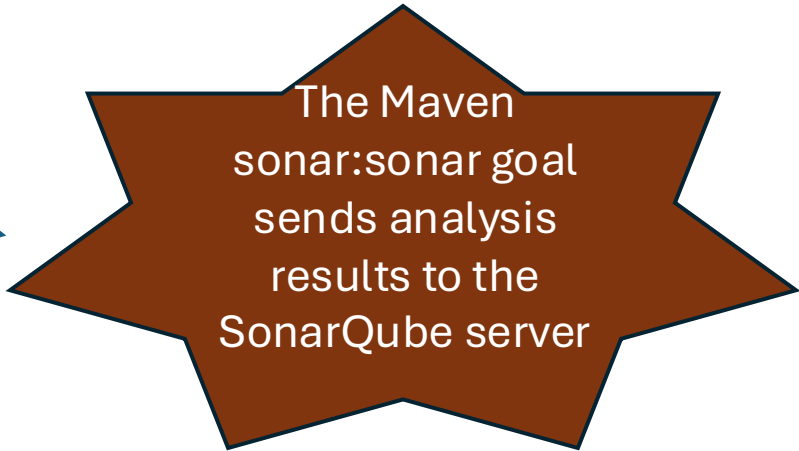
```
stage('Maven Build') {
  steps {
    echo 'Running Maven build...'
    bat 'mvn clean install'
  }
}
```



Cleans and
compiles the code,
runs tests, and
builds the package
(typically a .jar).



```
stage('SonarQube Analysis') {
  steps {
    echo 'Running SonarQube Analysis...'
    withSonarQubeEnv("${env.SONARQUBE_SERVER}") {
      bat 'mvn sonar:sonar'
    }
  }
}
```



The Maven
sonar:sonar goal
sends analysis
results to the
SonarQube server



```
stage('Quality Gate') {
  steps {
    echo 'Checking Quality Gate...'
    // This stage is usually used with waitForQualityGate() if SonarQube webhook is configured
    // bat 'mvn verify' could also go here if needed
  }
}
```

```

post {
    success {
        echo 'Build Successful: Archiving results and publishing tests.'
        junit '**/target/surefire-reports/TEST-*.xml'
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
    }

    failure {
        echo 'Build Failed!'
    }
}

```

Publishes JUnit test results.
Archives the JAR file built by Maven, making it downloadable from Jenkins UI.

Add the above code as the pipeline script and apply and save.

Then click on Build now...

jenkins-sonar-project-1

Stage View

		Declarative: Tool Install	Checkout	Maven Build	SonarQube Analysis	Quality Gate	Declarative: Post Actions
Average stage times:		306ms	6s	1min 15s	21s	253ms	364ms
#1	Jul 21 15:06 No Changes	306ms	6s	1min 15s	21s	253ms	364ms

9. Go to the SonarQube and see the results

Projects

Issues

Rules

Quality Profiles

Quality Gates

Administration

More

My Favorites

All

Filters

Quality Gate

Passed

Failed

Search projects (minimum 2 characters)

Perspective

Overall Status

Sort by

Name

☆ xmlFileEditor

Public

Last analysis: 3 minutes ago

92 Lines of Code

XML, Java

A 0

A 0

A 5

A —

0.0%

0.0%

Security

Reliability

Maintainability

Hotspots Reviewed

Coverage

Duplications

main

92 Lines of Code

Version 1.0-SNAPSHOT

Set as homepage

Quality Gate

Passed

New Code

Overall Code

Security

0 Open issues

A

Reliability

0 Open issues

A

Maintainability

5 Open issues

A

Accepted issues

0

Valid issues that were not fixed

Coverage

0.0%

On 18 lines to cover.

Duplications

0.0%

On 106 lines.

Security Hotspots

0

A

Bulk Change

Select issues

Navigate to issue

5 issues

35min effort

src/main/java/sjp/msc/cs/App.java

Replace this use of System.out by a logger.

Maintainability

Medium

Open

Not assigned

Adaptability

bad-practice

cert

L18

10min effort

1 year ago

src/main/java/sjp/msc/cs/Car.java

Remove this unused "model" private field.

Maintainability

Medium

Open

Not assigned

Intentionality

unused

End!